

Perform Multiple Linear Regression

```
1 from google.colab import drive
2 drive.mount('/content/drive')
```

Mounted at /content/drive

```
1 filepath = '/content/drive/MyDrive/DHF 3mtt Projects/Marketing Sales Data/marketing_sales_data.csv'
```

Step 1 - Importing modules and packages

```
1 # Import libraries and modules.
2
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 import statsmodels.api as sm
7 from statsmodels.formula.api import ols
```

```
1 df = pd.read_csv(filepath)
```

```
1 df.head()
```

	TV	Radio	Social Media	Influencer	Sales
0	Low	3.518070	2.293790	Micro	55.261284
1	Low	7.756876	2.572287	Mega	67.574904
2	High	20.348988	1.227180	Micro	272.250108
3	Medium	20.108487	2.728374	Mega	195.102176
4	High	31.653200	7.776978	Nano	273.960377

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
1 df.tail()
```

	TV	Radio	Social Media	Influencer	Sales
567	Medium	14.656633	3.817980	Micro	191.521266
568	High	28.110171	7.358169	Mega	297.626731
569	Medium	11.401084	5.818697	Nano	145.416851
570	Medium	21.119991	5.703028	Macro	209.326830
571	Low	13.221237	3.660566	Micro	135.773151

```
1 df.describe()
```

	Radio	Social Media	Sales
count	572.000000	572.000000	572.000000
mean	17.520616	3.333803	189.296908
std	9.290933	2.238378	89.871581
min	0.109106	0.000031	33.509810
25%	10.699556	1.585549	118.718722
50%	17.149517	3.150111	184.005362
75%	24.606396	4.730408	264.500118
max	42.271579	11.403625	357.788195

```
1 # Display number of rows, number of columns.
2 df.shape
```

```
(572, 5)
```

```
1 # Which columns contain null values and how many
2
3 df.isna().sum()
```

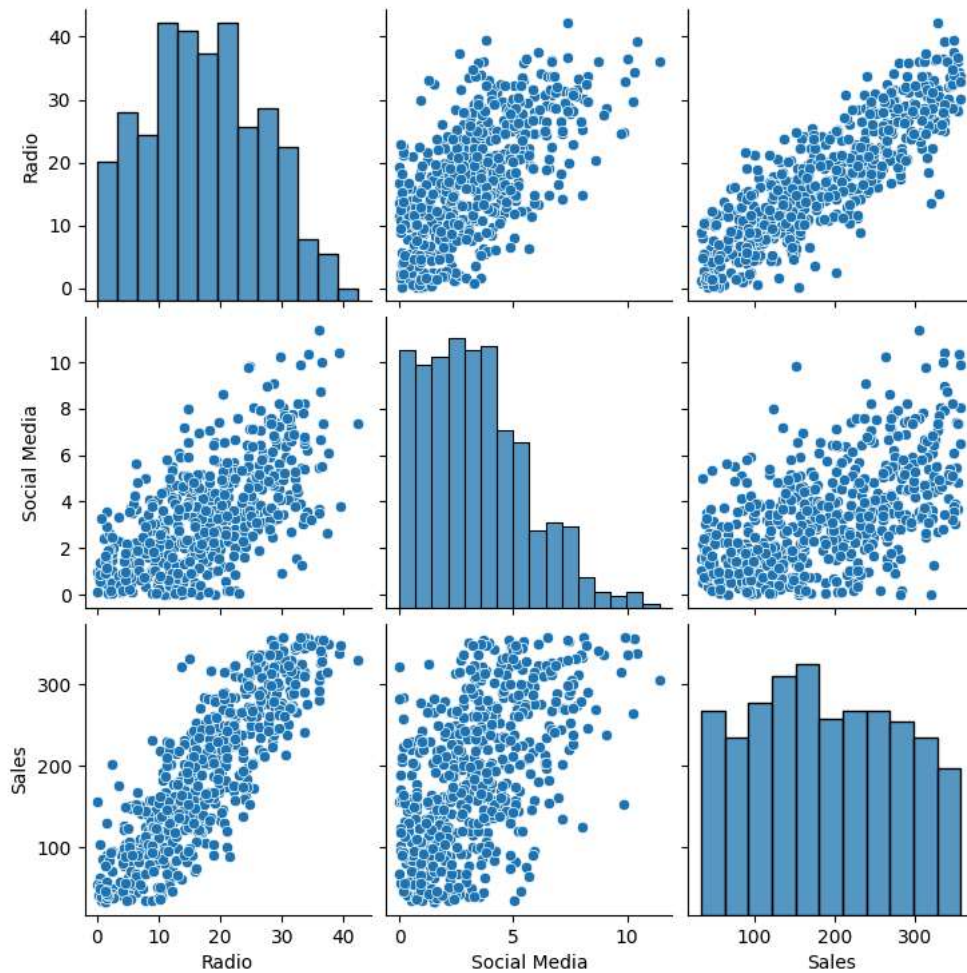
	0
TV	0
Radio	0
Social Media	0
Influencer	0
Sales	0

dtype: int64

Based on the validation check performed, the dataset does not contain any missing values or empty rows. This indicates that all records are complete, with no null entries or blank observations across the data structure.

✓ Step 2 - Exploratory Data Analysis

```
1 sns.pairplot(df);
```



Key insights:

Radio and Social Media both appear to have linear relationships with Sales. Given this, Radio and Social Media may be useful as independent variables in a multiple linear regression model estimating Sales.

TV and Influencer are excluded from the pairplot because they are not numeric.

Calculate the mean sales for each categorical variable

```
1 # Calculate the mean sales for each TV category.
2 print(df.groupby('TV')['Sales'].mean())
3
4 print('')
```

```

5
6 # Calculate the mean sales for each Influencer category .
7
8 print(df.groupby('Influencer')['Sales'].mean())

```

```

TV
High      300.853195
Low       90.984101
Medium    195.358032
Name: Sales, dtype: float64

```

```

Influencer
Macro     181.670070
Mega      194.487941
Micro     188.321846
Nano      191.874432
Name: Sales, dtype: float64

```

Key insights:

The average Sales for High TV promotions is considerably higher than for Medium and Low TV promotions. TV may be a strong predictor of Sales. The categories for Influencer have different average Sales, but the variation is not substantial. Influencer may be a weak predictor of Sales.

```

1 # Rename all columns in data that contain a space.
2
3 data = df.rename(columns={'Social Media': 'Social_Media'})

```

✓ Step 3 - Model Building

Fit a multiple linear regression model that predicts sales

```

1 # Define the OLS formula.
2
3 ols_formula = 'Sales ~ C(TV) + Radio'
4
5 # Create an OLS model.
6
7 OLS = ols(formula = ols_formula, data = data)
8
9 # Fit the model.
10
11 model = OLS.fit()
12
13 # Save the results summary.
14
15 model_results = model.summary()
16

```

```

17 # Display the model results.
18
19 model_results

```

```

OLS Regression Results

Dep. Variable: Sales      R-squared: 0.904
Model: OLS              Adj. R-squared: 0.904
Method: Least Squares   F-statistic: 1783.
Date: Thu, 25 Jun 2026  Prob (F-statistic): 1.63e-288
Time: 12:41:17          Log-Likelihood: -2714.0
No. Observations: 572    AIC: 5436.
Df Residuals: 568        BIC: 5453.
Df Model: 3

Covariance Type: nonrobust

   coef    std err   t    P>|t| [0.025   0.975]
Intercept    218.5261    6.261   34.902   0.000   206.228   230.824
C(TV)[T.Low]  -154.2971    4.929  -31.303   0.000  -163.979  -144.616
C(TV)[T.Medium] -75.3120    3.624  -20.780   0.000  -82.431  -68.193
Radio         2.9669    0.212   14.015   0.000    2.551    3.383

Omnibus: 61.244   Durbin-Watson: 1.870
Prob(Omnibus): 0.000   Jarque-Bera (JB): 18.077
Skew: 0.046       Prob(JB): 0.000119
Kurtosis: 2.134     Cond. No.    142.

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Key insights:

- TV was selected, as the preceding analysis showed a strong relationship between the TV promotional budget and the average Sales.
- Radio was selected because the pairplot showed a strong linear relationship between Radio and Sales. Social Media was not selected because it did not increase model performance and it was later determined to be correlated with another independent variable: Radio.
- Influencer was not selected because it did not show a strong relationship to Sales in the preceding analysis.

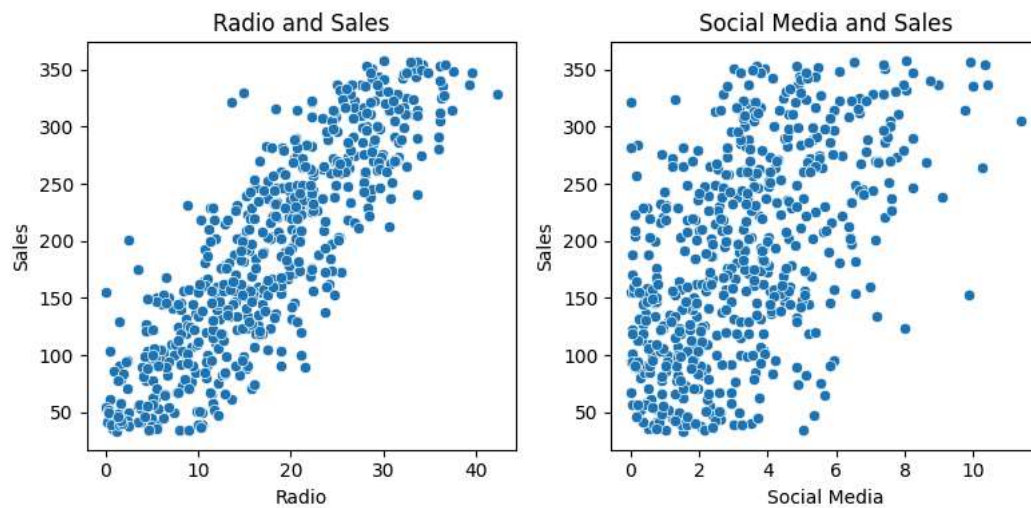
Check model assumptions

Model assumption: Linearity

```

1 # Create a scatterplot for each independent variable and the dependent variable.
2
3 # Create a 1x2 plot figure.
4 fig, axes = plt.subplots(1, 2, figsize = (8,4))
5
6 # Create a scatterplot between Radio and Sales.
7 sns.scatterplot(x = data['Radio'], y = data['Sales'],ax=axes[0])
8
9 # Set the title of the first plot.
10 axes[0].set_title("Radio and Sales")
11
12 # Create a scatterplot between Social Media and Sales.
13 sns.scatterplot(x = data['Social_Media'], y = data['Sales'],ax=axes[1])
14
15 # Set the title of the second plot.
16 axes[1].set_title("Social Media and Sales")
17
18 # Set the xlabel of the second plot.
19 axes[1].set_xlabel("Social Media")
20
21 # Use matplotlib's tight_layout() function to add space between plots for a cleaner appearance.
22 plt.tight_layout()

```



Key insight:

The linearity assumption holds for Radio, as there is a clear linear relationship in the scatterplot between Radio and Sales. Social Media was not included in the preceding multiple linear regression model, but it does appear to have a linear relationship with Sales.

Model assumption: Independence

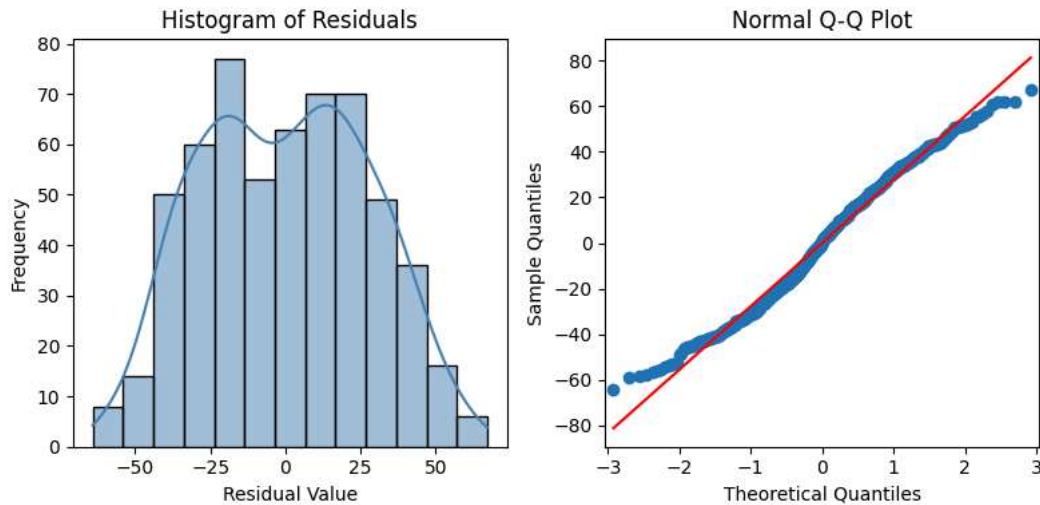
The independent observation assumption states that each observation in the dataset is independent. As each marketing promotion (i.e., row) is independent from one another, the independence assumption is not violated.

Model assumption: Normality

I create the following plots to check the normality assumption:

- Plot 1: Histogram of the residuals
- Plot 2: Q-Q plot of the residuals

```
1 # Extract residuals from the fitted model
2 residuals = model.resid
3
4 # Initialize a figure with two subplots (1 row, 2 columns)
5 fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(8, 4))
6
7 # --- Residual Histogram ---
8 sns.histplot(residuals, ax=axes[0], kde=True, color="steelblue")
9 axes[0].set_xlabel("Residual Value", fontsize=10)
10 axes[0].set_ylabel("Frequency", fontsize=10)
11 axes[0].set_title("Histogram of Residuals", fontsize=12)
12
13 # --- Q-Q Plot ---
14 sm.qqplot(residuals, line="s", ax=axes[1])
15 axes[1].set_title("Normal Q-Q Plot", fontsize=12)
16
17 # Optimize layout for readability
18 plt.tight_layout()
19
20 # Render plots
21 plt.show()
```

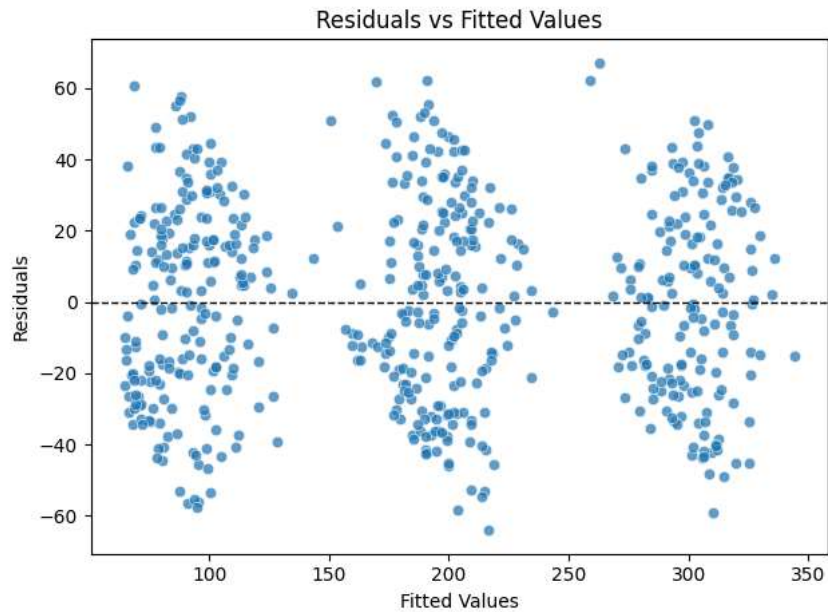
Key insight:

The histogram of the residuals are approximately normally distributed, which supports that the normality assumption is met for this model. The residuals in the Q-Q plot form a straight line, further supporting that this assumption is met.

Model assumption: Constant variance

I need to check that the constant variance assumption is not violated by creating a scatterplot with the fitted values and residuals.

```
1 # --- Residual Diagnostics: Fitted Values vs Residuals ---
2
3 # Generate scatterplot of fitted values against residuals
4 ax = sns.scatterplot(x=model.fittedvalues, y=model.resid, alpha=0.7)
5
6 # Configure axis labels and plot title
7 ax.set_xlabel("Fitted Values", fontsize=10)
8 ax.set_ylabel("Residuals", fontsize=10)
9 ax.set_title("Residuals vs Fitted Values", fontsize=12)
10
11 # Add horizontal reference line at y = 0
12 ax.axhline(0, color="black", linestyle="--", linewidth=1)
13
14 # Optimize layout and render plot
15 plt.tight_layout()
16 plt.show()
17
```



Key insights:

The fitted values are in three groups because the categorical variable is dominating in this model, meaning that TV is the biggest factor that decides the sales.

However, the variance where there are fitted values is similarly distributed, validating that the assumption is met.

Model assumption: No multicollinearity

The no multicollinearity assumption states that no two independent variables (X_i and X_j) can be highly correlated with each other.

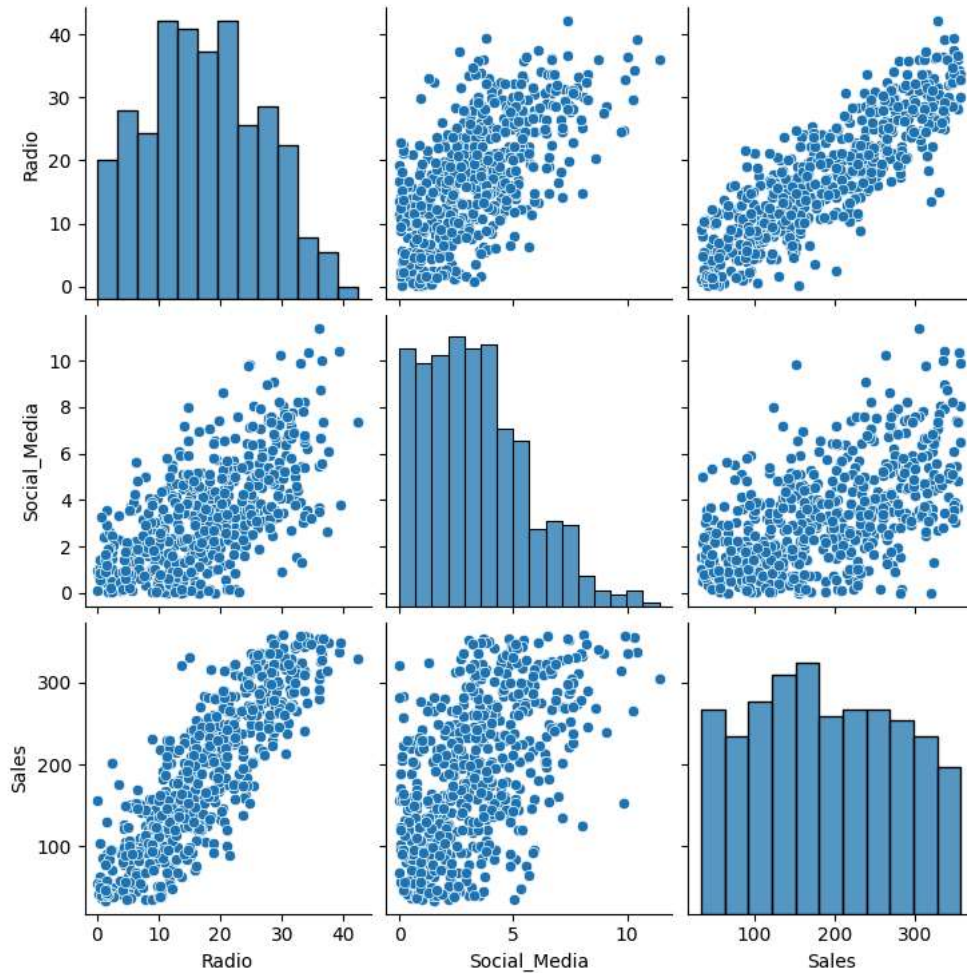
Two common ways to check for multicollinearity are to:

- Create scatterplots to show the relationship between pairs of independent variables
- Use the variance inflation factor to detect multicollinearity

I use both methods to check your model's no multicollinearity assumption.

```
1 # Create a pairplot of the data.  
2  
3 sns.pairplot(data)
```

<seaborn.axisgrid.PairGrid at 0x7f3c55d991f0>



```
1 # Calculate the variance inflation factor.
2
3 # Import variance_inflation_factor from statsmodels.
4 from statsmodels.stats.outliers_influence import variance_inflation_factor
5
6 # Create a subset of the data with the continuous independent variables.
7 X = data[['Radio', 'Social_Media']]
8
9 # Calculate the variance inflation factor for each variable.
10 vif = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
11
12 # Create a DataFrame with the VIF results for the column names in X.
13 df_vif = pd.DataFrame(vif, index=X.columns, columns = ['VIF'])
14
```

```
15 # Display the VIF results.
16 df_vif
```

	VIF	
Radio	5.170922	
Social_Media	5.170922	

Next steps: [Generate code with df_vif](#) [New interactive sheet](#)

Key insights:

The preceding model only has one continuous independent variable, meaning there are no multicollinearity issues.

If a model used both Radio and Social_Media as predictors, there would be a moderate linear relationship between Radio and Social_Media that violates the multicollinearity assumption. Furthermore, the variance inflation factor when both Radio and Social_Media are included in the model is 5.17 for each variable, indicating high multicollinearity.

✓ Step 4: Results and evaluation

Display the OLS regression results

```
1 # Display the model results summary.
2
3 model_results
```

OLS Regression Results

Dep. Variable:	Sales	R-squared:	0.904
Model:	OLS	Adj. R-squared:	0.904
Method:	Least Squares	F-statistic:	1783.
Date:	Thu, 25 Jun 2026	Prob (F-statistic):	1.63e-288
Time:	12:41:17	Log-Likelihood:	-2714.0

Key insight:

- Using TV and Radio as the independent variables results in a multiple linear regression model with $R^2 = 0.904$. In other words, the model explains 90.4% of the variation in Sales. This makes the model an excellent predictor of Sales.

Covariance Type: nonrobust

Interpret model coefficients

	coef	std err	t	P> t	[0.025	0.975]
Intercept	218.5261	6.261	34.902	0.000	206.228	230.824

```
1 # Display the model results summary.  
2  
3 model_results
```

Omnibus: 61.244 Durbin-Watson: 1.870
Prob(Omnibus): 0.000 Jarque-Bera (JB): 18.077
Dep. Variable: Sales R-squared: 0.904
Skew: 0.046 Prob(JB): 0.000119
Model: OLS Adj. R-squared: 0.904
Kurtosis: 2.134 Cond. No. 142
Method: Least Squares F-statistic: 1783.
Date: Thu, 25 Jun 2026 Prob (F-statistic): 1.63e-288
Time: 12:41:17 Log-Likelihood: -2714.0

Notes:

[1] Standard errors assume that the covariance matrix of the errors is correctly specified.

No. Observations:	572	AIC:	5436.
Df Residuals:	568	BIC:	5453.
Df Model:	3		